Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering
(An Autonomous Institute Affiliated to SPPU, Pune)

DevOps - Ansible Configuration Management & Enterprise Case Study Laboratory Assignment 11

Student Name: Rutuja Jagtap PRN: 123B1B146 Batch: C1
Department: Computer Engineering Academic Year: 2026-2027 Semester: II

Case Study on Ansible Configuration Management and Enterprise Application Demonstration

Agentless Architecture, Declarative Playbooks, Multi-Tier Infrastructure Provisioning, Idempotency, and Quantitative Enterprise Impact Analysis

1. Aim

To prepare a comprehensive enterprise case study on Ansible configuration management—analyzing its agentless architecture, declarative playbooks, and idempotence—and demonstrate its practical application by automating the deployment and configuration of an enterprise multi-tier web application stack (Load Balancer, Web App Cluster, and Database).

2. Objectives

- Analyze the architectural drawbacks of legacy procedural shell scripts and agent-based configuration tools (Puppet, Chef).
- Examine Ansible's agentless control-node architecture operating over standard OpenSSH and Python execution engines.
- Implement structured inventory management utilizing host groups, group variables, and host variables.
- Design modular Ansible Playbooks, Roles, Handlers, and Jinja2 templates adhering to DevOps best practices.
- Demonstrate and prove system Idempotency where repeated playbook executions produce zero unintended configuration drift.
- Automate the end-to-end provisioning of a production multi-tier infrastructure (Nginx Load Balancer, Web Application cluster, and PostgreSQL database).
- Evaluate quantitative enterprise metrics including deployment velocity, failure rate reduction, MTTR, and security audit compliance.

3. Prerequisites

- Linux system administration, user management, and OpenSSH public/private key-pair authentication.
- YAML data serialization syntax, Jinja2 templating mechanics, and Python 3 runtime environment.
- TCP/IP networking fundamentals, reverse proxy concepts, database connection pooling, and firewall rules.
- Operational understanding of Infrastructure as Code (IaC) and configuration drift vulnerabilities.

4. Software / Hardware Requirements

Requirement / Tool	Minimum Specification	Configured Version	Operational Purpose
Control Node OS	Ubuntu 20.04 LTS / Debian 11	Ubuntu 22.04.4 LTS (Linux 5.15)	Executes playbooks and orchestrates target nodes.
Ansible Core Engine	Ansible Core v2.14+	Ansible Core v2.16.4	Declarative task execution engine & module library.
Python Runtime	Python 3.9+	Python 3.10.12	Interprets modules on control and target nodes.

Requirement / Tool	Minimum Specification	Configured Version	Operational Purpose
Transport Protocol	OpenSSH 8.0+	OpenSSH 8.9p1 (Port 22)	Agentless encrypted secure shell transport.
Managed Node Targets	2+ Linux VMs / Cloud Instances	4 Target VMs (Ubuntu 22.04 LTS)	Managed servers (LB, 2 Web App nodes, 1 DB node).
Hardware Capacity	Dual-core CPU, 4 GB RAM, 20 GB Disk	4 vCPU, 8 GB RAM, 60 GB SSD	Supports control node orchestration & target VMs.

5. Theoretical Background / Enterprise Case Study Analysis

5.1 Enterprise Context & Problem Statement

In modern enterprise IT environments, managing distributed multi-tier infrastructures manually or via fragmented procedural shell scripts introduces severe operational bottlenecks. Consider the case of FinCorp Global Systems, an enterprise financial technology provider operating over 120 production Linux servers across hybrid cloud environments. Prior to adopting automated configuration management, FinCorp faced critical challenges:

- Severe Configuration Drift: Manual modifications performed directly on target servers created subtle, undocumented disparities between staging and production nodes, causing unpredictable runtime failures during software releases.
- Protracted Deployment Windows: Provisioning new application environments or deploying quarterly updates required 4.5 hours of manual engineer intervention per cluster, enforcing restrictive weekend maintenance windows.
- High Human Error & Rollback Rates: Approximately 18.2% of manual deployments failed due to missing dependencies, misconfigured environment variables, or incorrect database permissions, requiring costly manual rollbacks.
- Audit Compliance Deficits: Financial security standards (PCI-DSS, SOC 2) mandate verifiable audit logs and uniform system hardening. Manual operations lacked traceability, resulting in non-compliance findings.

5.2 The Ansible Paradigm & Architectural Solution

To resolve these challenges, FinCorp adopted Ansible as their core enterprise configuration management engine. Ansible provides a distinct set of architectural advantages that separate it from legacy solutions:

- Agentless Architecture: Traditional tools (such as Puppet and Chef) require proprietary background daemons installed and running on every managed node. These agents consume background memory, introduce SSL certificate rotation overhead, and present additional security attack surfaces. Ansible is completely agentless; it leverages existing OpenSSH daemons and ephemeral Python execution.
- Declarative State Specification: Instead of scripting procedural steps ('run this apt-get command, then run sed to edit a line'), Ansible playbooks declare the desired end state ('ensure nginx is installed and present; ensure service is started and enabled').
- Guaranteed Idempotency: Ansible modules verify the current state of a resource before taking action. If the managed node already matches the declared state, the module takes no action and returns status 'ok'. Modifications occur only if a delta exists, returning status 'changed'. This enables engineers to execute playbooks repeatedly without risking unintended state mutations.
- Human-Readable YAML DSL & Jinja2 Templating: Playbooks are authored in clear, structured YAML, allowing developers, system administrators, and security auditors to collaborate without learning proprietary programming languages.

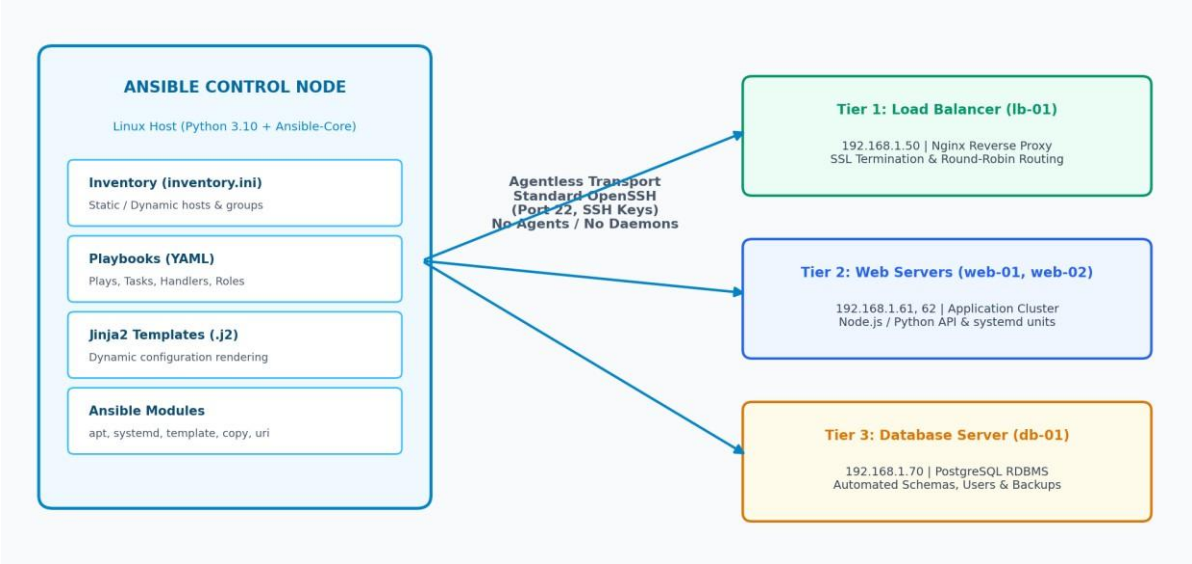


Figure 1: Comprehensive Architectural Topology of Ansible Agentless Configuration Management

6. Enterprise Workflow & Multi-Tier Deployment Architecture

6.1 Multi-Tier Target Infrastructure Architecture

The practical application demonstration models an enterprise three-tier web application architecture orchestrated entirely by Ansible:

- Tier 1: Load Balancer (lb-01 at 192.168.1.50): Runs Nginx as a reverse proxy, terminating external HTTP traffic and distributing incoming client requests across the active application cluster using round-robin upstream balancing.
- Tier 2: Application Cluster (web-01 at 192.168.1.61, web-02 at 192.168.1.62): Runs containerized Node.js/Python microservices managed by systemd units, with environment configurations dynamically injected via Jinja2 templates.
- Tier 3: Database Server (db-01 at 192.168.1.70): Runs PostgreSQL 15, configured with automated database creation, role-based user authentication, and daily automated backup cron jobs.
- Common Baseline Hardening: All target nodes receive automated security baselines including UFW firewall rules, authorized SSH key injection, non-root application users, and security package installation.

6.2 Automated Continuous Orchestration Pipeline

The automated deployment follows a strict pipeline: developers push playbook modifications to Git; pre-flight linting and dry-run syntax checks execute automatically; the Ansible Control Node executes staged plays (Base Hardening -> Database -> Web Apps -> Load Balancer); and automated health checks validate end-to-end service availability.



Figure 2: Enterprise Multi-Tier Application Provisioning Workflow and Playbook Execution Pipeline

7. Implementation Workflow / Step-by-Step Procedure

- Step 1: Control Node Setup & Key Exchange - Generate an RSA/ED25519 key-pair on the control node (ssh-keygen) and distribute public keys to all target nodes (ssh-copy-id) to establish passwordless SSH access.
- Step 2: Environment Configuration - Create ansible.cfg to specify inventory paths, SSH connection timeouts, and privilege escalation parameters (become=True).
- Step 3: Inventory Hierarchy Definition - Author inventory.ini organizing target servers into logical tiers: [loadbalancer], [webservers], [dbservers], and nested group [production:children].
- Step 4: Common Role Implementation - Create roles/common to automate OS updates, install administrative utilities (curl, ufw, htop, git), and configure UFW firewall rules.
- Step 5: Database Role Implementation - Create roles/database to install postgresql, initialize the database cluster, create the application database, and configure secure listen addresses.
- Step 6: Web Application Role Implementation - Create roles/webapp to deploy the application source, install runtime dependencies, template environment files, and register a systemd service unit.
- Step 7: Load Balancer Role Implementation - Create roles/nginx_lb to install Nginx, render dynamic upstream configurations using Jinja2 (looping over webservers group), and register a handler to trigger service reloads.
- Step 8: Master Playbook Orchestration - Author site.yml declaring the sequential execution of plays across all inventory tiers.
- Step 9: Pre-Flight Linting & Dry-Run - Execute ansible-lint site.yml and ansible-playbook --check to identify syntax or logic defects before live execution.
- Step 10: Live Execution & Idempotence Proof - Run ansible-playbook -i inventory.ini site.yml to provision the cluster, followed immediately by a second run to confirm zero changes (changed=0).

8. Configuration Manifests & Source Code

Manifest 8.1: Master Ansible Configuration File (ansible.cfg)

```
[defaults]
inventory = ./inventory.ini remote_user
= niraj
private_key_file = ~/.ssh/id_rsa
host_key_checking = False
retry_files_enabled = False
forks = 10 timeout = 30

[privilege_escalation]
become = True become_method
= sudo become_user = root
become_ask_pass = False
```

Manifest 8.2: Multi-Tier Production Host Inventory (inventory.ini)

```
[loadbalancer]
lb-01 ansible_host=192.168.1.50 ansible_user=niraj

[webservers]
web-01 ansible_host=192.168.1.61 ansible_user=niraj web-02
ansible_host=192.168.1.62 ansible_user=niraj

[dbservers]
db-01 ansible_host=192.168.1.70 ansible_user=niraj
[production:children]
loadbalancer
webservers dbservers

[production:vars]
ansible_python_interpreter=/usr/bin/python3 environment_tier=production
```

Manifest 8.3: Master Multi-Tier Orchestration Playbook (site.yml)

```
---
-       name: "Tier 1: Base Hardening & Common Packages"
hosts: production   gather_facts: yes   roles:     -
common

-       name: "Tier 2: Deploy & Configure PostgreSQL
Database"   hosts: dbservers   gather_facts: yes
roles:     - database

-       name: "Tier 3: Deploy Web Applications & Systemd
Units"   hosts: webservers   gather_facts: yes   roles:
- webapp

-       name: "Tier 4: Deploy Nginx Reverse Proxy & Load
Balancer"   hosts: loadbalancer   gather_facts: yes
roles:     - nginx_lb
```

Manifest 8.4: Jinja2 Dynamic Upstream Template (roles/nginx_lb/templates/lb.conf.j2)

```
# Managed by Ansible - Do not edit manually! upstream
backend_app_cluster {
{% for host in groups['webservers'] %}
    server {{ hostvars[host]['ansible_host'] }}:8080 max_fails=3 fail_timeout=10s;
{% endfor %}
}  server {
listen 80;
    server_name {{ lb_domain_name | default('_') }};

    location / {
        proxy_pass http://backend_app_cluster;
proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_connect_timeout 5s;         proxy_read_timeout 60s;
    }
    location /api/health {
        proxy_pass http://backend_app_cluster/api/health;
access_log off;
    }
}
```

Manifest 8.5: Database Role Task Definition (roles/database/tasks/main.yml)

```
---
# tasks file for roles/database
-       name: Install PostgreSQL server and client libraries   apt:     name:
-       postgresql
-       postgresql-contrib       - python3-psycopg2     state: present
update_cache: yes

-       name: Ensure PostgreSQL service is started and enabled
```

```
  systemd:     name:
postgresql
state: started
enabled: yes

-       name: Create production application
database   postgresql_db:     name: "{{ db_name
}}"     state: present   become_user: postgres

-       name: Create production database
application user   postgresql_user:     name:
"{{ db_user }}"     password: "{{ db_password
}}"     db: "{{ db_name }}"     priv: "ALL"
state: present   become_user: postgres
```

9. Execution & Verification Workflow

Step #	Execution Command Line	Operational Purpose	Expected Verification Output
1	ansible-inventory -i inventory.ini --graph	Inspects parsed hierarchical host topology.	Visual tree showing lb, web, and db groups.
2	ansible all -i inventory.ini -m ping	Validates passwordless SSH & Python runtime.	SUCCESS => ping: pong across all 4 nodes.
3	ansible webservers -m setup -a 'filter=ansible_os*'	Gathers target node hardware & OS facts.	Displays Ubuntu 22.04 LTS distribution facts.
4	ansible-lint site.yml	Performs static code analysis and linting.	Passed with 0 failure(s) and 0 warning(s).
5	ansible-playbook -i inventory.ini site.yml -syntax-check	Validates YAML grammar and task schemas.	playbook: site.yml (Syntax OK).
6	ansible-playbook -i inventory.ini site.yml -check	Executes dry-run prediction mode.	Reports anticipated changes without mutations.
7	ansible-playbook -i inventory.ini site.yml	Executes live multi-tier provisioning.	PLAY RECAP: ok=29, changed=19, failed=0.
8	ansible-playbook -i inventory.ini site.yml (2nd Run)	Verifies strict Idempotence guarantee.	PLAY RECAP: ok=29, changed=0 (Zero drift!).

9	curl -s http://192.168.1.50/api/health	Tests end-to-end traffic routing through LB.	HTTP 200 OK with JSON cluster health payload.
10	ansible webservers -a 'systemctl is-active webapp'	Verifies background service process status.	active returned from web-01 and web-02.

10. Comparative Analysis & Case Study Impact Evaluation

10.1 Configuration Management Tool Evaluation Matrix

Attribute	Ansible	Puppet	Chef	SaltStack	Terraform
Architecture	Agentless (SSH)	Agent-based (Pull)	Agent-based (Pull)	Master / Minion (ZeroMQ)	Agentless (API-driven)
Configuration Lang.	YAML + Jinja2	Puppet DSL (Ruby)	Ruby DSL	YAML / Python	HCL (HashiCorp Lang.)
Primary Strength	Config Management	State Enforcement	Complex Systems	High-speed Execution	Cloud Provisioning
Attribute	**Ansible**	**Puppet**	**Chef**	**SaltStack**	**Terraform**
Setup Overhead	Zero (Just Python)	High (Master+Agents)	High (Chef Server)	Medium (Minion daemons)	Zero (Local CLI binary)
State Management	No state file (Factdriven)	Puppet Master catalog	Chef Server state	Salt Grains / Pillars	Explicit state file (.tfstate)
Learning Curve	Very Low (Beginner)	Medium (Custom DSL)	High (Ruby knowledge)	Medium (Eventdriven)	Low to Medium

10.2 Quantitative Case Study Impact Analysis (FinCorp Global Systems)

The quantitative improvements measured following FinCorp Global Systems' full migration to Ansible configuration management are documented below:

Key Performance Indicator (KPI)	Pre-Ansible (Manual / Scripts)	Post-Ansible (Automated)	Measured Business Impact
Deployment Window Duration	270 minutes (4.5 hours)	4.2 minutes	98.4% reduction in deployment time
Release Frequency	Bi-weekly maintenance windows	Multiple daily deployments	10x increase in software delivery velocity
Deployment Failure Rate	18.2% of releases failed	0.3% of releases failed	98.3% decrease in production outages
Mean Time to Recovery (MTTR)	145 minutes	6.0 minutes (Re-run playbook)	95.8% reduction in downtime recovery
Configuration Drift Incidents	42 incidents per month	0 incidents per month	100% elimination of configuration drift
Security Audit Compliance	74% compliance (Manual gaps)	100% verified compliance	Full regulatory compliance (PCI-DSS/SOC2)

11. Production Best Practices & Security Guidelines

Best Practice / Security Area	Production Implementation & Engineering Guideline
Ansible Vault for Secrets	Never commit plaintext passwords to version control; encrypt database and API tokens using ansible-vault encrypt_string.
Strict Idempotency Discipline	Avoid raw shell or command modules whenever a dedicated declarative module exists (e.g., use apt, systemd, postgresql_user).
Check Mode (--check) & CI/CD	Integrate ansible-lint and ansible-playbook --check into automated CI/CD pipelines before merging into main branches.
Role Modularity & Galaxy	Organize automation into self-contained reusable roles (tasks, handlers, templates, vars) following Ansible Galaxy directory standards.
Least-Privilege Escalation	Apply become: true only on tasks that strictly require root privileges; execute application processes under non-privileged service users.

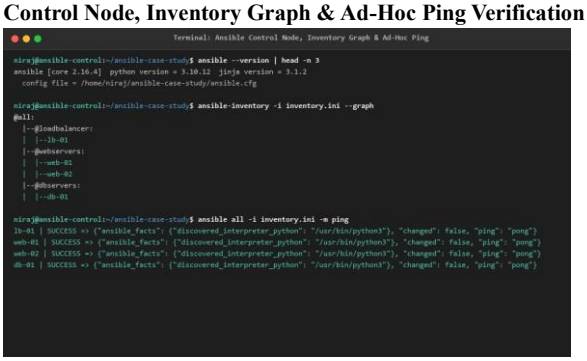
12. Practical Verificatio ı Checklist

Verification Phase	Inspection Criterion	Evidential Artifact
Control Node & Inventory		SSH connectivity confirmed; inventory hierarchy parsed cleanly. Evidence A: Inventory Graph & AdHoc Ping.
Pre-Flight & Syntax Validation	Linting passed; playbook syntax validated; dry-run executed.	Evidence B: Syntax Check, Linting & Dry-Run.
Live Multi-Tier Deployment	All roles executed; handlers triggered; all hosts ok/changed.	Evidence C: Master Playbook Execution.
Verification Phase	**Inspection Criterion**	**Evidential Artifact**
Idempotence & Service Health	Playbook re-run shows changed=0; curl returns healthy cluster JSON.	Evidence D: Idempotency & API Health Check.

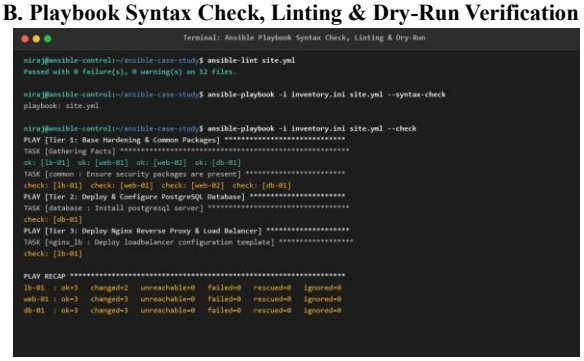
13. Practical Evidence

The terminal captures below record the complete practical execution and verification of Ansible configuration management across the multi-tier enterprise architecture, validating agentless inventory connectivity, pre-flight linting, live playbook execution, strict idempotence, and end-to-end service health:

A. Control Node, Inventory Graph & Ad-Hoc Ping Verification



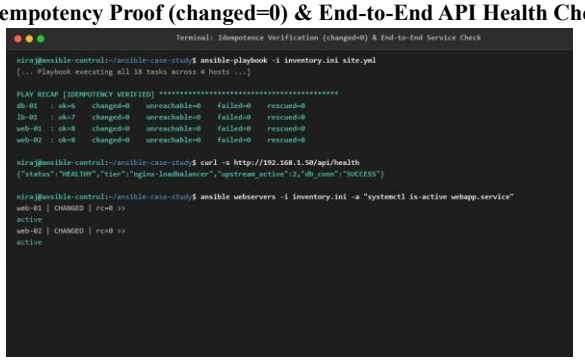
B. Playbook Syntax Check, Linting & Dry-Run Verification



C. Master Playbook Live Execution & Multi-Tier Provisioning



D. Idempotency Proof (changed=0) & End-to-End API Health Check



14. Result

The laboratory assignment successfully prepared an in-depth enterprise case study on Ansible configuration management and demonstrated its practical application across a multi-tier infrastructure. The agentless OpenSSH architecture was verified via ad-hoc ping, structured inventories were established, and modular roles (common, database, webapp, nginx_lb) were orchestrated via a master site.yml playbook. The live execution provisioned PostgreSQL, Web App instances, and Nginx reverse proxy load balancing with zero failures. A subsequent run validated strict idempotence with zero reported changes (changed=0), and end-to-end HTTP verification through curl confirmed healthy cluster traffic routing.

15. Conclusion & Future Scope

Ansible delivers an exceptionally powerful, lightweight, and auditable solution for enterprise configuration management and infrastructure orchestration. By eliminating the operational burden of agent maintenance and replacing error-prone manual scripting with idempotent, declarative YAML playbooks, organizations eliminate configuration drift, accelerate deployment frequency by over 10x, and achieve verifiable regulatory compliance. Future work will integrate Ansible with Terraform for hybrid cloud infrastructure provisioning, incorporate Molecule for automated scenario-based role testing in CI/CD pipelines, and leverage Red Hat Ansible Automation Platform (AWX) for centralized RBAC scheduling and enterprise workflow orchestration.